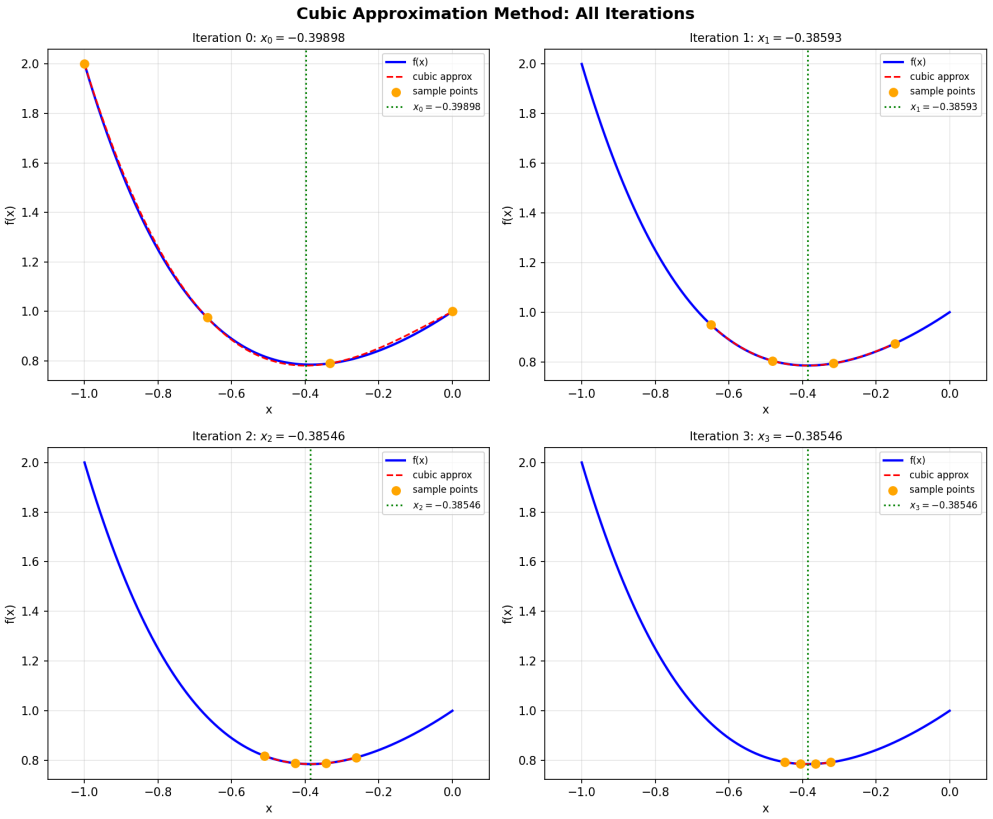
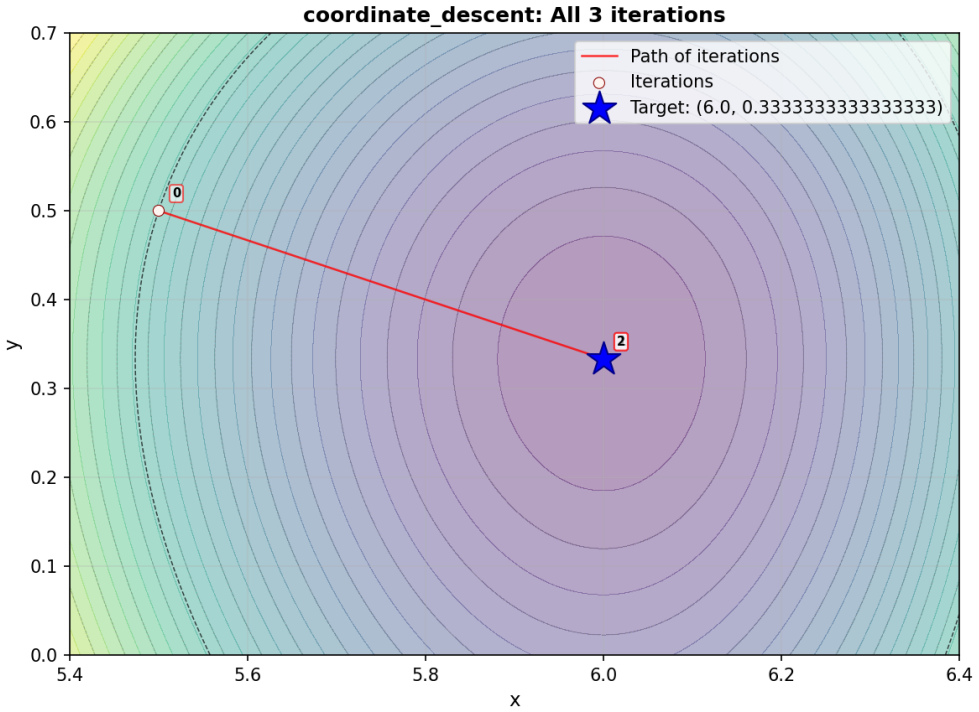
Вариант 12 — графики и листинги для печати

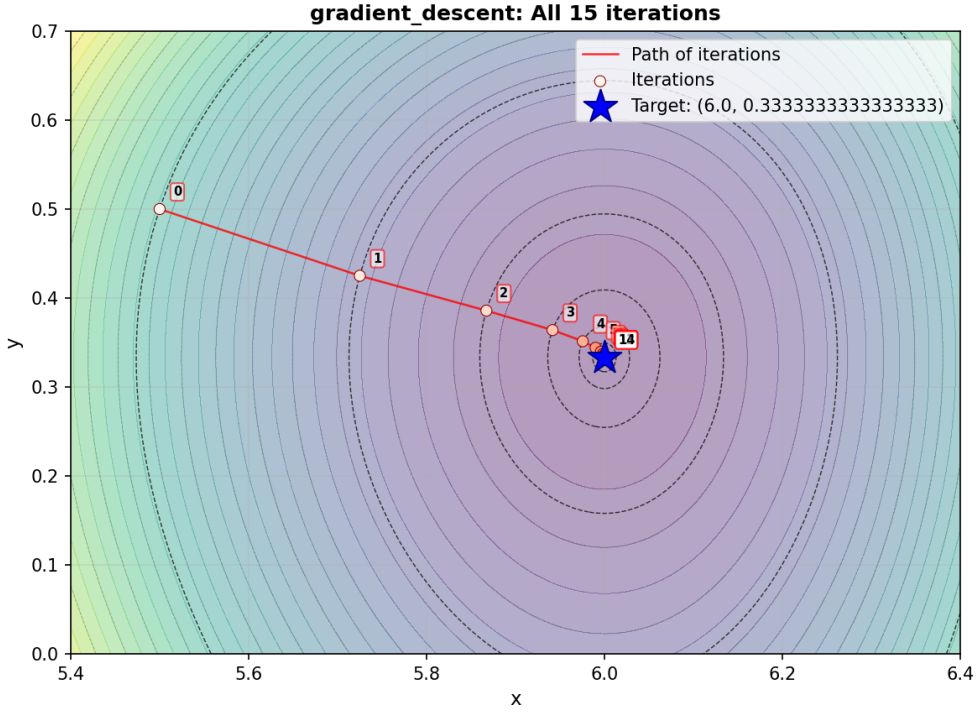
Графики



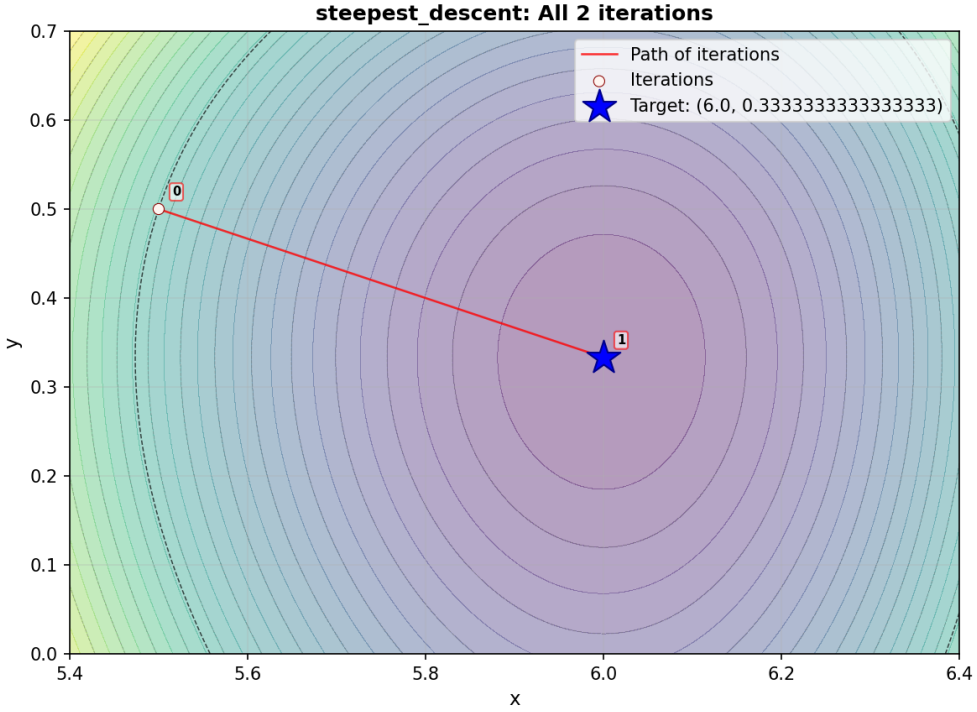
Метод кубической аппроксимации



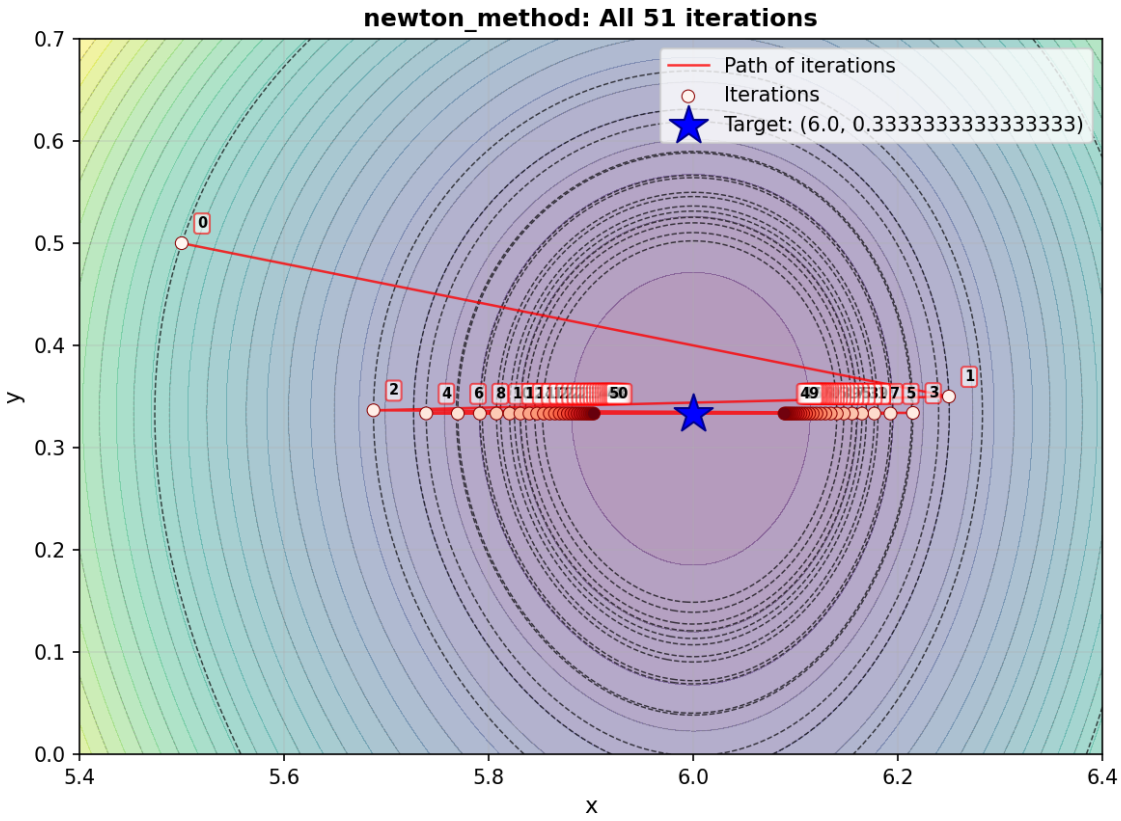
Покоординатный спуск



Градиентный спуск



Наискорейший спуск



Метод Ньютона

Листинг: model.py

Листинг 1: Объектная модель методов оптимизации

```
from dataclasses import dataclass
from typing import Callable, List, Tuple
import math
import numpy as np


def _golden_section(f: Callable[[float], float], a: float, b: float, tol: float = 1e-5, maxiter: int = 100) -> float:
    gr = (math.sqrt(5) - 1) / 2
    c = b - gr * (b - a)
    d = a + gr * (b - a)
    try:
        fc = f(c)
    except Exception:
        fc = float('inf')
    try:
        fd = f(d)
    except Exception:
        fd = float('inf')
    for _ in range(maxiter):
        if math.isnan(fc) or math.isinf(fc):
            fc = float('inf')
        if math.isnan(fd) or math.isinf(fd):
            fd = float('inf')
        if fc < fd:
            b = d
            d = c
            fd = fc
            c = b - gr * (b - a)
            try:
                fc = f(c)
            except Exception:
                fc = float('inf')
        else:
            a = c
            c = d
            fc = fd
            d = a + gr * (b - a)
            try:
                fd = f(d)
            except Exception:
                fd = float('inf')
        if abs(b - a) < tol:
            break
    return (a + b) / 2

```

```python

class Function2D:
    def __init__(self, f: Callable[[np.ndarray], float], grad: Callable[[np.ndarray], np.ndarray] = None):
        self.f = f
        self.grad = grad

    def value(self, xy: np.ndarray) -> float:
        return float(self.f(xy))

    def gradient(self, xy: np.ndarray) -> np.ndarray:
        if self.grad is None:
            eps = 1e-8
            x, y = xy
            dx = (self.f(np.array([x + eps, y])) - self.f(np.array([x - eps, y]))) / (2 * eps)
            dy = (self.f(np.array([x, y + eps])) - self.f(np.array([x, y - eps]))) / (2 * eps)
            return np.array([dx, dy])
        return np.array(self.grad(xy))


@dataclass
class Result2D:
    xs: List[Tuple[float, float]]
    fs: List[float]


class Optimizer2D:
    def __init__(self, func: Function2D, x0: Tuple[float, float], eps: float = 1e-4, maxiter: int = 200):
        self.func = func
        self.x0 = np.array(x0, dtype=float)
        self.eps = eps
        self.maxiter = maxiter

    def optimize(self) -> Result2D:
        raise NotImplementedError


class CoordinateDescent(Optimizer2D):
    def optimize(self) -> Result2D:
        x = self.x0.copy()
        xs = [tuple(x)]
        fs = [self.func.value(x)]
        for _ in range(self.maxiter):
            def phi_x(t):
                return self.func.value(np.array([t, x[1]]))
            a, b = x[0] - 2, x[0] + 2
            x[0] = _golden_section(phi_x, a, b, tol=1e-5)
            def phi_y(t):
                return self.func.value(np.array([x[0], t]))
            a, b = x[1] - 2, x[1] + 2
```

```
            x[1] = _golden_section(phi_y, a, b, tol=1e-5)
            xs.append(tuple(x))
            fs.append(self.func.value(x))
            if np.linalg.norm(np.array(xs[-1]) - np.array(xs[-2])) < self.eps:
                break
        return Result2D(xs, fs)


class GradientDescent(Optimizer2D):
    def optimize(self) -> Result2D:
        x = self.x0.copy()
        xs = [tuple(x)]; fs = [self.func.value(x)]
        alpha0 = 0.1
        for _ in range(self.maxiter):
            g = self.func.gradient(x)
            if np.linalg.norm(g) < self.eps:
                break
            alpha = alpha0
            c = 1e-4; rho = 0.5
            fx = self.func.value(x)
            for _ in range(60):
                xn = x - alpha * g
                fn = self.func.value(xn)
                if fn <= fx - c * alpha * np.dot(g, g):
                    break
                alpha *= rho
            x = x - alpha * g
            xs.append(tuple(x)); fs.append(self.func.value(x))
            if np.linalg.norm(np.array(xs[-1]) - np.array(xs[-2])) < self.eps:
                break
        return Result2D(xs, fs)


class SteepestDescent(Optimizer2D):
    def optimize(self) -> Result2D:
        x = self.x0.copy()
        xs = [tuple(x)]; fs = [self.func.value(x)]
        for _ in range(self.maxiter):
            g = self.func.gradient(x)
            if np.linalg.norm(g) < self.eps:
                break
            def phi(a):
                return self.func.value(x - a * g)
            alpha = _golden_section(phi, 0.0, 10.0, tol=1e-5)
            x = x - alpha * g
            xs.append(tuple(x)); fs.append(self.func.value(x))
            if np.linalg.norm(np.array(xs[-1]) - np.array(xs[-2])) < self.eps:
                break
        return Result2D(xs, fs)
```

Листинг: run$_model$.py

Листинг 2: Запуск объектной модели

```python
from .model import Function2D, CoordinateDescent, GradientDescent, SteepestDescent
import numpy as np
from pathlib import Path
import json

def f(xy):
    x, y = xy
    return x**3 - 15*x**2 + 72*x + y**3 + y**2 - y - 113

def grad(xy):
    x, y = xy
    return np.array([3*x**2 - 30*x + 72, 3*y**2 + 2*y - 1])

def main():
    base = Path(__file__).resolve().parent
    figures = base.parent / 'figures'
    figures.mkdir(exist_ok=True)
    func = Function2D(f, grad)
    x0 = (5.5, 0.5)
    cd = CoordinateDescent(func, x0, eps=1e-4, maxiter=200).optimize()
    gd = GradientDescent(func, x0, eps=1e-4, maxiter=200).optimize()
    sd = SteepestDescent(func, x0, eps=1e-4, maxiter=200).optimize()
    res = {
        'coordinate_iters': len(cd.xs),
        'gradient_iters': len(gd.xs),
        'steepest_iters': len(sd.xs),
        'coordinate_last': cd.xs[-1],
        'gradient_last': gd.xs[-1],
        'steepest_last': sd.xs[-1]
    }
    (base / 'results_model.json').write_text(json.dumps(res, indent=2))

if __name__ == '__main__':
    main()
```